

Large-Scale Book Recommendation System

SDS 2412 — Analysis of Large Datasets

Team Members:

Rodney Okoth

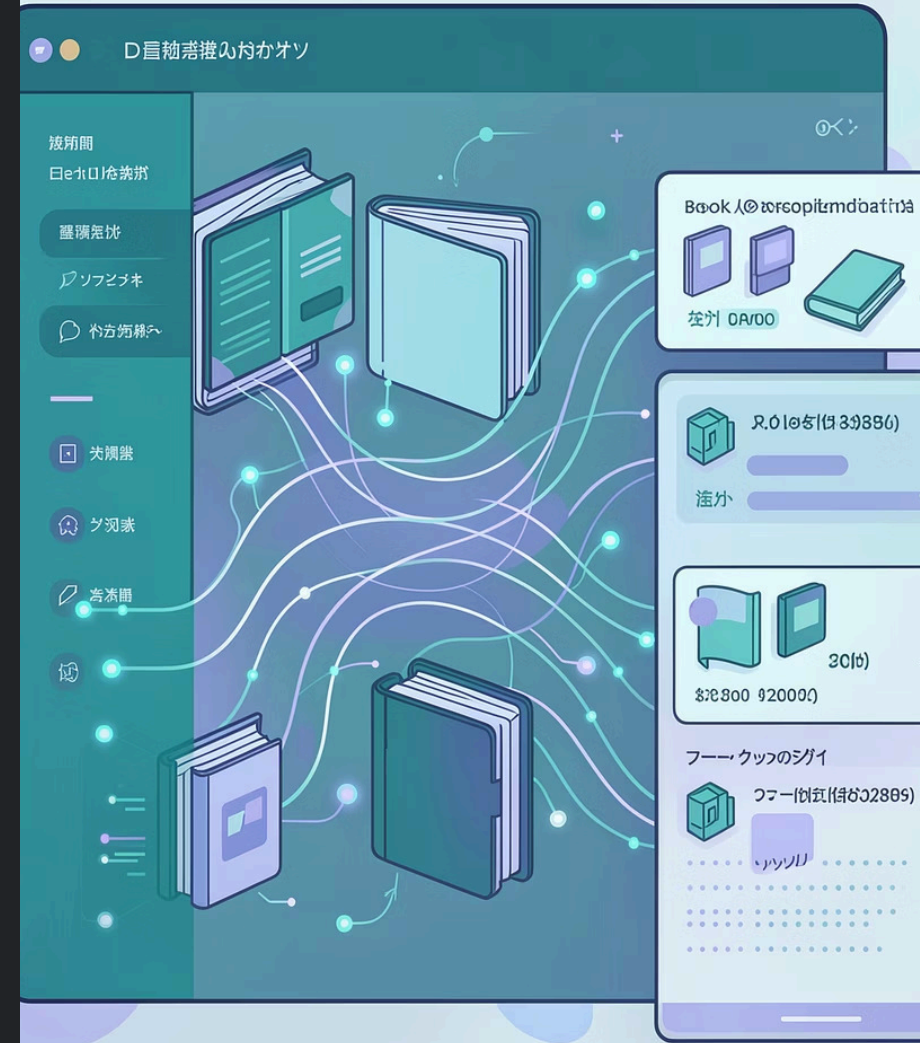
Sandra Jebet

Effie Auma

Leslie Gideon

Kandy Genga

May 2026



Why Recommendations Are Hard

The Book-Crossing dataset presents a fundamental challenge: extreme sparsity. With 1.1 million ratings across 270,000 books and 278,000 users, the user-item matrix is 99.8% empty. Most users have rated almost nothing, and 70% of all ratings are implicit zeros rather than genuine opinions. The system must find signal in an almost entirely empty dataset.

1.1M

Total Ratings

433,000 explicit ratings used for
training

99.8%

Matrix Sparsity

The core challenge of all
recommendation systems

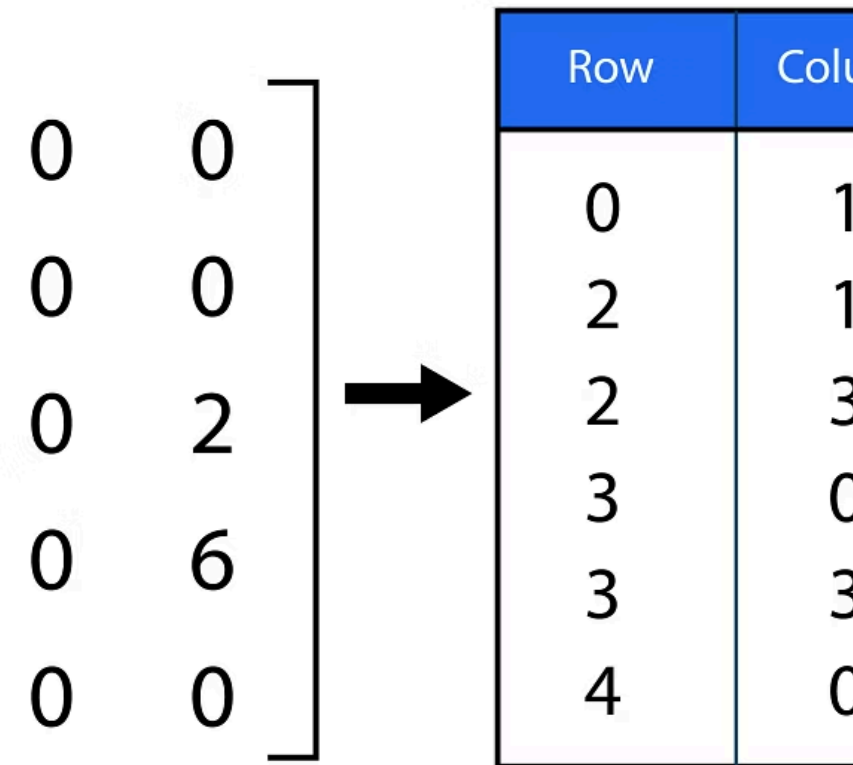
70%

Implicit Ratings

Zeros separated from real ratings
before modeling

Goal: Predict which books a user will genuinely like before they discover them.

Sparse Matrix



Row	Col
0	1
2	1
2	3
3	0
3	3
4	0

Lambda Architecture

Three Layers, One System

We chose Lambda Architecture because it solves the accuracy versus speed trade-off. Batch processes everything accurately on a schedule. Speed processes live events instantly but approximately. The serving layer merges both.



Batch Layer

Spark processes all 433K ratings — accurate, scheduled



Speed Layer

Stream simulator at 15,000 events/sec — instant, approximate



ML Layer

Hybrid ALS + TF-IDF model trained on batch outputs



Serving Layer

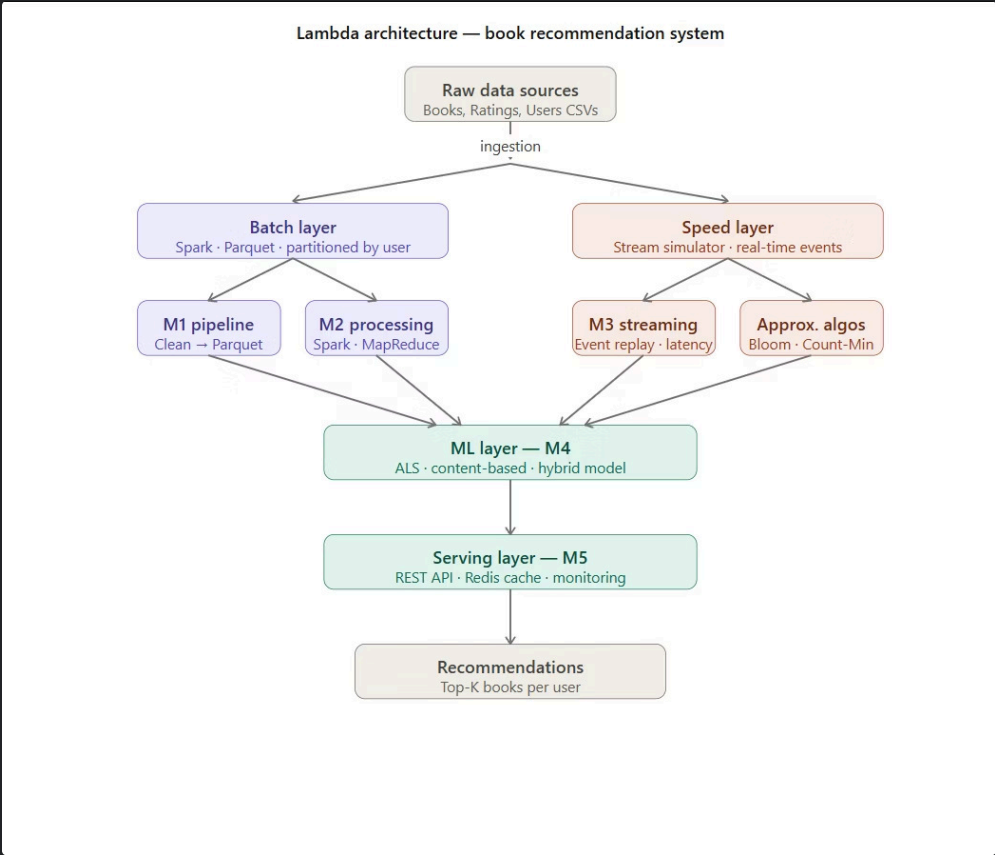
REST API with LRU cache delivering Top-K recommendations

✔ **Innovation:** MMR diversity injection preventing filter bubbles

Milestone 1 & 2: Data & Distributed Processing

From Raw CSVs to a Spark Cluster

M1 cleaned three raw CSVs and saved everything to Parquet — columnar storage, 5–10x faster than CSV. M2 moved the data into Spark, sharded it into 10 partitions by User-ID, and ran MapReduce batch jobs to build user profiles and item popularity tables that feed the ML layer.



Implicit vs Explicit

70% of ratings are score=0 — separated before training



Cleaning

Fixed year column corruption, filled missing ages with median, extracted country from location strings



Parquet

Columnar storage — 5–10x faster and smaller than CSV



Sharding

10 Spark partitions by User-ID — balanced, no data skew



MapReduce

User profiles and item popularity computed in parallel



Scalability

Near-linear scaling confirmed — 0.75s for 433K rows, projected 4.3s for 25M on 10 nodes

Milestone 3: Streaming & Real-Time

The Speed Layer

Designed for Apache Kafka — simulated in Python due to notebook environment constraints. The processing logic is architecturally identical. Events are processed one by one, trending books tracked in a sliding window, and Bloom filters give $O(1)$ lookups for already-read books.

1

Kafka Design

Simulated with Python event loop — same processing logic

2

Throughput

15,000 events per second

3

Latency

0.016ms average per event

4

Sliding Window

Last 500 events in memory — old events dropped automatically

5

Bloom Filters

625 bytes per user, 40x more memory efficient than dictionaries, $O(1)$ lookup

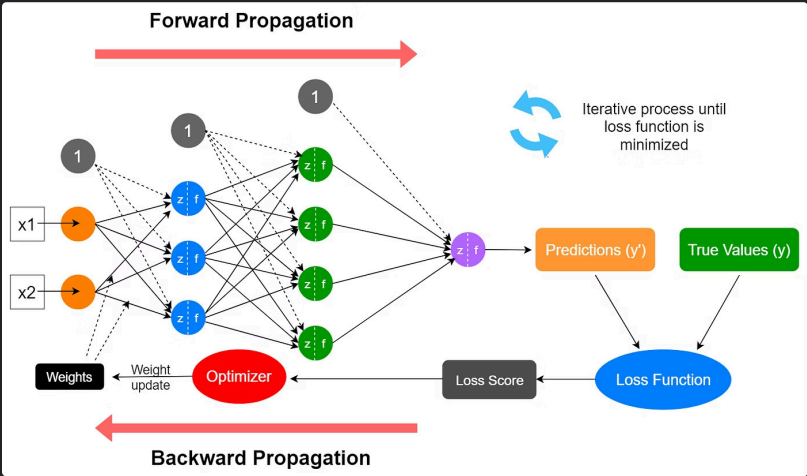
Batch vs Streaming: Batch is exact but hours old. Streaming is approximate but live. Lambda uses both.

Milestone 4: Hybrid ML Engine

Three Models, One Recommender

ALS learns from rating patterns — who rated what. TF-IDF learns from book content — author, publisher, year. The hybrid combines both. Tested on a real user who rated Harry Potter.

01	02
Collaborative Filtering (ALS)	Content-Based
Matrix factorization, 50 latent factors, trained in Spark	TF-IDF vectors, 271K books × 5K features, cosine similarity
03	
Hybrid	
70% ALS + 30% content-based — ALS dominates where data is rich, content-based covers sparse books	



2.487

RMSE

Noisy dataset, expected result

1.954

MAE

Mean absolute error

0.86

Precision@10

8.6 out of 10 recommendations are books the user rated 7 or above

Key insight: High Precision@10 despite moderate RMSE — ranking quality is what matters for recommendations

Milestone 5: Optimization & Deployment

Making the System Production-Ready

M5 adds three production components: caching so repeat requests never touch the model, a real HTTP API so any application can request recommendations, and a drift monitor that watches prediction error and alerts when the model needs retraining.



LRU Cache

60% hit rate — cached requests return in under 1ms versus hundreds of milliseconds uncached



Flask REST API

/recommend returns personalized books as JSON — /health reports system status



Drift Monitor

Rolling MAE window of 100 predictions — stable at 1.99, no retraining triggered

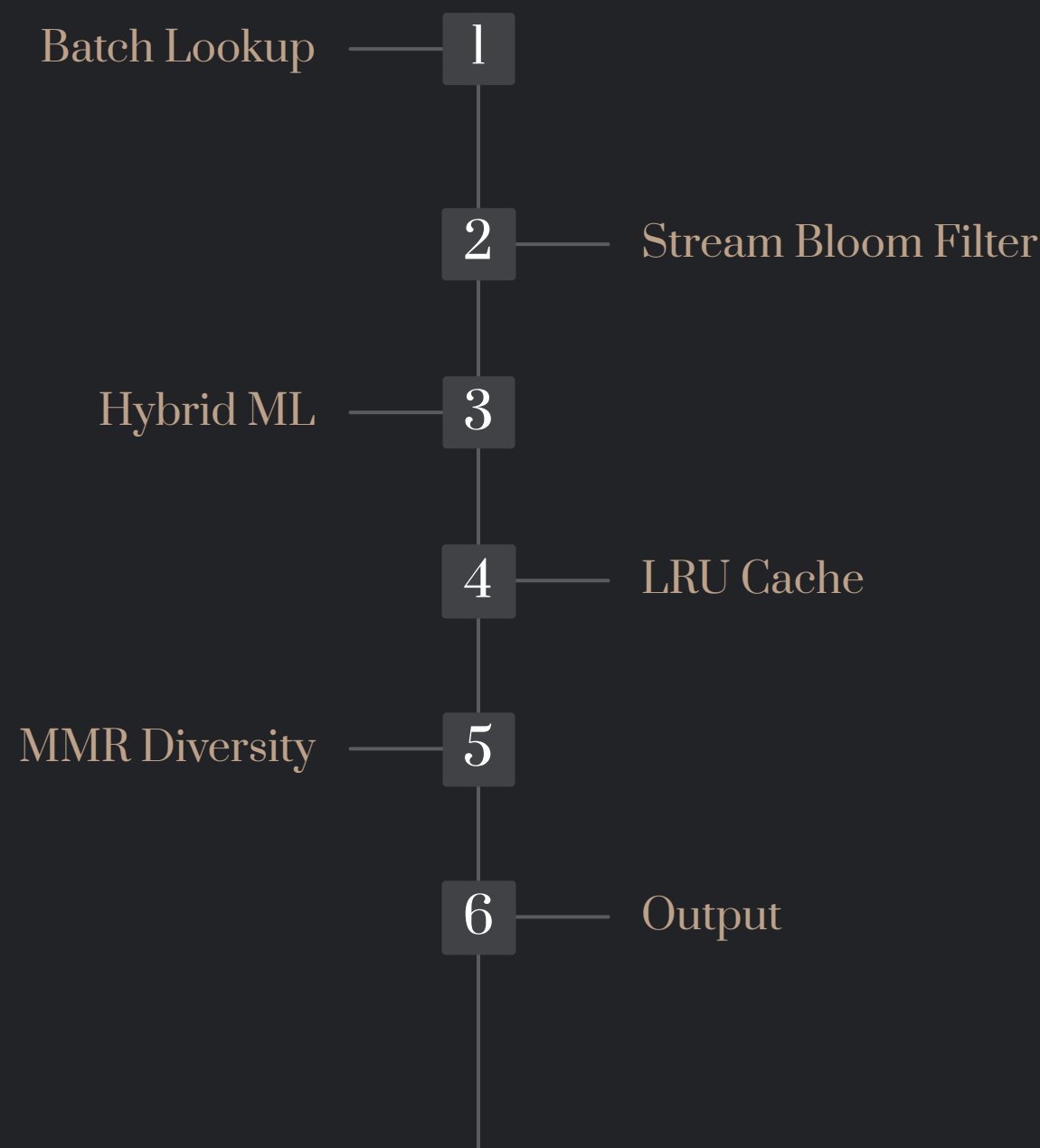
Production path: Cache → Redis, user profiles → MongoDB



Milestone 6: Full Integration & Capstone

Everything Connected, End to End

One pipeline function activates all five previous milestones in sequence for a single user request. We also added MMR diversity injection — our system-level innovation — which prevents the filter bubble problem where pure collaborative filtering keeps recommending the same popular authors.



MMR Innovation

Penalizes repeated authors by 0.3 — guarantees variety in top 10 without losing relevance

Evaluation

20 users tested end-to-end across all layers simultaneously

Scale

50 Spark nodes → batch in ~10 seconds, stream at 750K events/sec — same code, bigger infrastructure

Failures & Resolutions

What Failed and How We Fixed It

Evaluation Trap

Problem: Precision@10 looped 200 separate Spark queries — extremely slow

Solution: Fixed by pulling all predictions into pandas in one batch job

Schema Corruption

Problem: Year column had text mixed in

Solution: Fixed with `pd.to_numeric(errors='coerce')`

Missing Data

Problem: Dropping null ages would lose 40% of users

Solution: Fixed with median fill

Kafka Constraint

Problem: Cannot run a Kafka broker in a notebook

Solution: Simulated with Python event loop, processing logic unchanged

Hybrid Signal

Problem: Content-based scores near zero for most results because seed book metadata rarely matched ALS recommendations

Solution: Documented honestly, ALS correctly dominates for active users

Conclusion

What We Built

An end-to-end large-scale recommendation system — from raw CSV files to a deployed REST API — built on production-grade tools and architecture. The same system on a real cluster with Kafka and 50 Spark nodes would serve millions of users with no code changes.

Lambda Architecture

Batch, streaming, ML, and serving layers fully integrated

Hybrid ML Engine

ALS collaborative filtering + TF-IDF content-based

Precision@10: 0.86

8.6 out of 10 recommendations genuinely liked by the user

LRU Cache

60% hit rate — most requests never touch the model

MMR Diversity Injection

Prevents filter bubbles across all recommendations

Flask REST API

With drift monitoring and health checks

"The same architecture on a real cluster would power a production recommendation engine serving millions of users — no structural changes required."